This section will show you how to set up a centralized configuration server with Spring Cloud Config that's responsible for delivering configuration data stored in a Git repository to all applications. Chapter 14 will cover more advanced configuration topics, including secrets management and Kubernetes features like ConfigMaps and Secrets. Many of the features and patterns you'll use with Spring Cloud Config are easily applicable to the other solutions involving configuration services and cloud vendor services.

NOTE Your choice of configuration service will depend on your infrastructure and requirements. For example, suppose you're running your workloads on Azure already, and you need a GUI to manage configuration data. In that case, it might make sense to use Azure Key Vault rather than running a configuration service yourself. If you want to version-control your configuration data with Git, Spring Cloud Config or Kubernetes ConfigMaps and Secrets would be a better choice. You might even compromise and use a managed Spring Cloud Config service offered by vendors like Azure or VMware Tanzu.

The idea of centralized configuration is built around two main components:

- A data store for configuration data, providing persistence, versioning, and possibly access control
- A server sitting on top of the data store to manage configuration data and serve it to multiple applications

Imagine having many applications deployed in different environments. A configuration server could manage configuration data for all of them from a centralized place, and that configuration data might be stored in different ways. For example, you could use a dedicated Git repository for storing non-sensitive data and use HashiCorp Vault to store your secrets. No matter how the data is stored, a configuration server will deliver it to different applications through a unified interface. Figure 4.7 shows how centralized configuration works.